

# Building a Homelab Patch Management Server

Foreman + Katello on RHEL 9: the full journey — installation, patch management setup, client registration, and everything that went wrong (and how it was fixed)

This guide is written in plain, everyday language. Wherever a technical word is unavoidable, it's explained the first time it comes up, and every term also appears in the Glossary right after this page — so if you forget what something means later, flip back here first.

Each problem below follows the same simple structure: **What we were trying to do**, **What went wrong**, **Why it happened**, **How we fixed it**, and **How to avoid or fix it if it happens again**.

# Glossary (plain-language explanations)

Read this section once, top to bottom, before the rest of the guide — it will make everything afterward much easier to follow.

## **Foreman**

A free program that lets you manage many Linux servers from one central website instead of logging into each one separately.

## **Katello**

An add-on to Foreman that specifically handles software updates/patches: downloading them once, organizing them, and deciding which servers get which updates and when.

## **Red Hat Satellite**

Red Hat's official, paid product that is built on top of Foreman and Katello. What we built ourselves is the free, open-source version of the same idea.

## **RHEL**

Red Hat Enterprise Linux — the specific flavor ("distribution") of Linux used on our server and client machines.

## **Homelab**

A personal, small-scale setup at home used for learning and practicing — not a real company environment.

## **VM (Virtual Machine)**

A computer that runs entirely inside software, on top of a real physical computer, so you can have several separate "computers" on one machine.

## **ESXi**

The software (from VMware) that lets one physical computer host and run several virtual machines.

## **Terminal / Shell**

The text-based window where you type commands directly to the computer, instead of clicking buttons.

## **SSH**

A secure way to remotely log into another computer's terminal over a network, as if you were sitting in front of it.

## **sudo**

A command that lets a regular user temporarily run something "as the administrator" (root), usually by re-entering their own password.

## **root**

The administrator account on Linux, with permission to do absolutely anything on the system.

## **Package / RPM**

A single installable piece of software, bundled up in a standard format (RPM) so Linux can install, update, or remove it cleanly.

## **Repository ("repo")**

An online storage location holding many packages, which your computer can browse and download from.

## **dnf**

The tool RHEL uses to install, update, and remove packages (similar to an "app store" command).

### **Module / Module stream**

A way RHEL offers a choice between different versions of the same software (for example, Ruby version 3.0 vs 3.1) so you can pick which one your system uses.

### **Ruby**

A programming language that Foreman and Katello are written in; certain versions of Ruby are required for certain versions of Foreman/Katello to work.

### **Puppet**

A tool that Foreman originally used to automatically configure and install all its pieces in the correct order.

### **Container**

A self-contained, sealed "box" that carries a program along with everything it needs to run, so it doesn't depend on whatever is already installed on the host computer.

### **Podman**

The program used to create, run, and manage containers on our RHEL server (similar in purpose to the more widely known "Docker").

### **foremanctl**

The newer, container-based tool we ultimately used to install Foreman and Katello, avoiding the Ruby version conflicts we hit with the older installation method.

### **systemd**

The background system on Linux that starts, stops, and monitors services (like the database, the web server, etc.) automatically.

### **Service**

A program that runs continuously in the background, such as the database or the web server.

### **PostgreSQL (Postgres)**

The database program that Foreman uses to store all of its information (organizations, hosts, packages, etc.).

### **Redis**

A small, fast support service that Foreman uses to temporarily store short-lived data while it's running.

### **SELinux**

A built-in RHEL security feature that restricts what each program is allowed to touch on the system, even if that program is otherwise compromised.

### **Volume**

A folder on the real server that a container is allowed to read from or write to, so its data survives even if the container itself is restarted.

### **EPEL**

"Extra Packages for Enterprise Linux" — a large, trusted, community-maintained collection of extra software not included in RHEL by default.

### **Content host**

A server that has been registered with Foreman/Katello, so it can receive updates and be tracked from the central dashboard.

**Product**

In Foreman, a folder that groups related repositories together (for example, "all repos related to RHEL 9").

**Content View**

A curated, frozen snapshot of one or more repositories — a specific, tested "release" of packages that you choose to hand out to servers, instead of whatever is newest at any given moment.

**Lifecycle Environment**

A named stage in your update rollout process (for example, "Development" then "Production"), so you can test updates before they reach your most important servers.

**Activation Key**

A pre-set bundle of instructions a new server presents when it registers with Foreman, telling it which organization, content view, and lifecycle environment it belongs to.

**Errata**

An official notice describing a specific update or fix (often for a security issue), including which packages it changes and why.

**CVE**

A unique reference number assigned to a specific, publicly known security vulnerability, so it can be tracked and referenced consistently everywhere.

**Hardening**

The general process of making a system more secure by turning off what's unnecessary and locking down what remains.

**OpenSCAP**

A tool built into RHEL that automatically checks your system against official security checklists and reports exactly what needs fixing.

**Organization / Location**

Ways Foreman groups and separates servers, useful for larger setups with multiple teams or physical sites (in a homelab, usually just "Default").

**hammer**

Foreman's command-line tool, letting you type commands instead of clicking through the website to do the same tasks.

**API**

A structured way for programs (or the register command you ran) to talk to Foreman directly, rather than through the visual website.

**Bearer token**

A temporary, secret code that proves you're allowed to do something (like register a server), similar in spirit to a temporary access pass.

**SSL certificate**

A digital "ID card" a website presents to prove it's genuine; a self-signed one is one your own server made for itself, rather than one issued by a widely-trusted outside authority — which is why browsers and some tools flag it as unverified by default.

**Self-signed certificate**

A security certificate that a server generated and "vouches for" itself, instead of getting it from a recognized outside authority. Perfectly fine for a private homelab, but tools are cautious about it by default.

### **DNS / hosts file**

The general system computers use to turn a name (like satellite.homelab.local) into a real network address. The "hosts file" is a manual, local override you can add on a single computer when there's no real DNS entry for a custom name.

### **IP address**

The numeric address (like 192.168.50.3) that identifies a specific computer on a network.

# What's in this guide

## Part 1: Installing Foreman and Katello

1. Shell variables not carrying over between commands
2. Missing EPEL repository on RHEL
3. Ruby version conflict (first appearance)
4. Interrupting the install partway through
5. Ruby 3.0 completely missing from the system
6. The real wall: Katello not yet ready for the newer Ruby
7. Switching to the container-based installer
8. Corrupted database data folder
9. A leftover program fighting with the container
10. Missing memory setting warning
11. Success, and where to find your login
12. Opening the site from a Windows computer

## Part 2: Setting Up Patch Management

13. Understanding what we're building and why
14. Creating the Product and Repository
15. A repository sync failing partway through
16. Content Views moved to a different menu

## Part 3: Registering a Client Server

17. Root password not accepted
18. Registration failing as "unauthorized"
19. Registration blocked by a security certificate check
20. Client thinking it's already registered

General lessons for next time

# Part 1: Installing Foreman and Katello

Setting up the central server itself, on a RHEL 9 virtual machine.

## 1. Shell variables not carrying over between commands

### What we were trying to do

Set two short-cut values (FOREMAN\_VERSION and OS\_MAJOR\_VERSION) so we wouldn't have to retype version numbers in every command.

### What went wrong

The download web addresses came back broken, with gaps where the version numbers should have been (like [releases/ell/x86\\_64](#)).

### Why it happened

The values were set in one terminal window, then used in a different one (for example, after closing and reopening the terminal). Once a terminal window closes, anything set inside it disappears.

### How we fixed it

We set the values again and immediately checked them before doing anything else, all in the same window:

```
export FOREMAN_VERSION=3.13
export OS_MAJOR_VERSION=9
echo $FOREMAN_VERSION
echo $OS_MAJOR_VERSION
```

**If this happens again:** Always set a value and use it right away, in the same terminal window. If you switch windows, set it again first.

## 2. Missing EPEL repository on RHEL

### What we were trying to do

Add the EPEL collection of extra software to the server.

### What went wrong

Running the usual install command for EPEL failed with "No match for argument" and "Unable to find a match".

### Why it happened

Real RHEL (unlike CentOS or Rocky Linux) doesn't include a simple installable shortcut for EPEL — it has to be added using the exact web address of the EPEL file.

### How we fixed it

We installed it using the direct link instead:

```
sudo dnf install -y https://dl.fedoraproject.org/pub/epel/epel-release-latest-9.noarch.rpm
```

**If this happens again:** On real RHEL, always add EPEL using its full web address, not just a short name.

## 3. Ruby version conflict (first appearance)

### What we were trying to do

Continue the installation after setting up EPEL and other required repositories.

### What went wrong

The installer failed partway, saying a package needed an older piece of software (rdoc) than what was currently available, which was "filtered out".

### Why it happened

RHEL lets you choose between different bundled versions of certain software (for example, Ruby 3.0 vs 3.1). We had the newer one selected, but some required pieces still needed the older one. Once you choose a version, RHEL refuses to touch pieces meant for the other version.

### How we fixed it

We checked which version was active, then fully switched back (not just "enabled", but properly switched, including anything already installed):

```
sudo dnf module list ruby --enabled
sudo dnf module switch-to ruby:3.0 -y
sudo dnf distro-sync -y
```

**If this happens again:** If you see the phrase "filtered out by modular filtering", it almost always means two versions of the same software are in conflict. Check with "dnf module list" and use "switch-to", not just "enable".

## 4. Interrupting the install partway through

### What we were trying to do

Let the long installation process finish.

### What went wrong

The install was going well (over 1000 out of about 1360 total steps done), but it was stopped early (using Ctrl+C) to check something else. The next attempt showed new problems that weren't there before.

### Why it happened

The installer sets up hundreds of connected pieces in one long run. Stopping it partway can leave some pieces set up and others not, which can look like brand-new problems on the next try.

### How we fixed it



The installer is designed to safely run again and again, so we simply let it run to the very end without interrupting it, using a second window to watch progress instead:

```
tail -f /var/log/foreman-installer/katello.log
```

**If this happens again:** Don't press Ctrl+C on a long install unless it's truly stuck. Open a second window to check progress instead.

## 5. Ruby 3.0 completely missing from the system

### What we were trying to do

Switch back to the older Ruby version to resolve the earlier conflict.

### What went wrong

This time, trying to switch back gave a hard error: "missing groups or modules: ruby:3.0" — the option to pick that version was gone entirely.

### Why it happened

RHEL periodically retires older bundled software versions from its update servers over time. By this point, Ruby 3.0 had been fully removed — only newer versions remained as options. You cannot install a version the update server no longer offers at all.

### How we fixed it

We confirmed this, and realized going backward was no longer possible — a different, forward-looking fix was needed instead (see the next two sections):

```
sudo dnf module list ruby
```

**If this happens again:** If a version has vanished completely from the list (not just "disabled"), stop trying to switch back to it. Look for a newer approach that matches what's currently available instead.

## 6. The real wall: Katello not yet ready for the newer Ruby

### What we were trying to do

Try a newer version of Foreman and Katello, hoping it would support the newer Ruby already on the system.

### What went wrong

Even the newer version still failed, with certain packages explicitly saying they "require Ruby older than 3.1".

### Why it happened

This wasn't a mistake on our part — it was a genuine, real-world gap between two projects moving at different speeds. RHEL had already moved its Ruby forward, but Katello's own code hadn't fully caught up yet for every piece.

## How we fixed it

We checked Foreman's official documentation and community forums, which confirmed this was a known, current limitation, and pointed to a different, better installation method (covered next).

**If this happens again:** If an error names an exact version limit and switching versions doesn't fix it, that's usually a real current limitation of the software. Check the official docs/forums for the currently recommended method rather than continuing to patch around it yourself.

## 7. Switching to the container-based installer

### What we were trying to do

Find an installation method that avoids fighting the server's Ruby version entirely.

### What went wrong

We switched to Foreman's newer, container-based installer, called `foremanctl`.

### Why it happened

This method runs Foreman and Katello inside sealed "containers", each one bundled with its own copy of Ruby. Because each piece brings its own Ruby, it no longer matters what version RHEL itself currently has.

### How we fixed it

We removed the old broken attempt, then installed and ran the new method:

```
sudo dnf remove -y foreman-installer-katello katello foreman-proxy foreman-postgresql \
foreman-redis foreman-dynflow-sidekiq
sudo rm -f /etc/yum.repos.d/foreman*.repo /etc/yum.repos.d/katello*.repo
/etc/yum.repos.d/puppet*.repo
sudo dnf clean all
sudo dnf copr enable @theforeman/foremanctl rhel-9-x86_64 -y
sudo dnf upgrade -y
sudo dnf install -y foremanctl
sudo foremanctl deploy
```

**If this happens again:** If a project offers both an older, traditional installer and a newer container-based one, and you're hitting deep version conflicts, try the container-based option. Note it may be labeled a preview/newer feature, meaning it's not yet the most polished official path, but it can be far more reliable for avoiding host system conflicts.

## 8. Corrupted database data folder

### What we were trying to do

Let the container-based install finish setting up the database.

### What went wrong

The database container failed to start, saying its data folder was "not a valid data directory" and was missing a required file.

## Why it happened

An earlier, failed install attempt had already partly written files into that same folder. The database found a folder that was neither empty (needed to set up fresh) nor a complete, working database (needed to just start normally) — stuck in between.

## How we fixed it

We stopped the affected service, completely emptied that folder, and let it start over cleanly:

```
sudo systemctl stop postgresql.service
sudo rm -rf /var/lib/pgsql/data/*
sudo restorecon -Rv /var/lib/pgsql/data/
sudo systemctl start postgresql.service
```

**If this happens again:** If you retry an install after an earlier failed attempt, and a database complains its folder is invalid or incomplete, it's usually safe (with no real data yet) to empty that folder and let it start fresh.

# 9. A leftover program fighting with the container

## What we were trying to do

Get past the final "waiting for Foreman to become accessible" step of the install.

## What went wrong

That step kept failing, and buried in the error was a message about a support service (Redis) being unable to save its data to disk.

## Why it happened

Earlier troubleshooting had accidentally installed a plain, ordinary copy of that same support service directly onto the operating system. This old copy grabbed control before the new, properly set-up container version could start, and the old copy didn't have the right folder permissions to save its data.

## How we fixed it

We removed the old, plain copy entirely and let the container-managed version take over:

```
sudo systemctl stop redis.service
sudo systemctl disable redis.service
sudo dnf remove -y redis
sudo systemctl daemon-reload
sudo systemctl reset-failed
sudo systemctl start redis.service
```

**If this happens again:** If a background service shows as "running" but never appears in your container list, it may be a plain, non-container copy quietly conflicting with the container version. Check with a package search ("rpm -qa | grep <name>") to see if one snuck in.

## 10. Missing memory setting warning

### What we were trying to do

Make sure the support service (Redis) runs safely going forward.

### What went wrong

Once we could see its logs clearly, there was a warning that a certain memory-related setting wasn't turned on — the exact type of setting connected to the problem we'd just fixed.

### Why it happened

That service occasionally saves a snapshot of its data in the background. Linux has a setting that must be explicitly turned on to allow this safely, and it wasn't on by default.

### How we fixed it

We turned it on immediately, no restart needed:

```
echo 'vm.overcommit_memory = 1' | sudo tee -a /etc/sysctl.conf  
sudo sysctl vm.overcommit_memory=1
```

**If this happens again:** Fix warnings in a service's logs right away, even if things seem to work for now — they're often an early sign of the same kind of failure you may have just spent time debugging.

## 11. Success, and where to find your login

### What we were trying to do

Finish the installation completely, with no remaining errors.

### What went wrong

After all the fixes above, the final run completed with zero failures.

### Why it happened

Every earlier blocker had been resolved, so the installer could finally get through every step cleanly.

### How we fixed it

The final output included a message with the actual login details, which should be saved somewhere safe right away:

```
Foreman is running at https://satellite.homelab.local  
Admin credentials: admin:YOUR-PASSWORD
```

**If this happens again:** As soon as a password like this is shown, copy it into a password manager immediately, then log in once and change it to something memorable.

## 12. Opening the site from a Windows computer

### What we were trying to do

Open the Foreman website from a regular Windows laptop/desktop, not the server itself.

### What went wrong

Typing the address into a browser either couldn't find the site, or timed out.

### Why it happened

The address ends in ".local", which isn't a real, public web address — it only works if you specifically tell your computer which numeric address it points to.

### How we fixed it

We found the server's real numeric address on the RHEL machine:

```
ip addr show | grep "inet " | grep -v 127.0.0.1
```

Then, on the Windows computer, opened Notepad as Administrator, and opened this exact file (typing the full path directly avoids Notepad hiding it):

```
C:\Windows\System32\drivers\etc\hosts
```

Added one line at the bottom with the real address, and saved it:

```
192.168.50.3 satellite.homelab.local
```

**If this happens again:** Any custom ".local" address needs this same manual entry on every computer that needs to reach it, not just the server itself.

## Part 2: Setting Up Patch Management

Turning the running server into an actual central patching hub.

### 13. Understanding what we're building and why

Instead of logging into every single Linux machine separately and running updates by hand, all your servers can register to this one central Foreman server. Foreman downloads updates once, and every registered server pulls its updates from Foreman instead of the open internet directly. This gives one dashboard showing exactly which servers are missing which updates, and lets you control the rollout (test on a few machines first, then the rest) instead of everyone updating whenever they happen to.

The pieces involved, explained simply:

- Repository = the actual shelf of packages (e.g. "every RHEL BaseOS package that exists")
- Product = a folder that groups related repositories together
- Content View = a frozen, tested snapshot of packages you've decided is safe to hand out
- Lifecycle Environment = a stage in your rollout pipeline, like Dev then Production
- Activation Key = the "onboarding paperwork" a new server presents when it registers

### 14. Creating the Product and Repository

#### What we were trying to do

Give Foreman an actual source of update packages to manage.

#### What went wrong

The repository was created successfully, but showed 0 packages, 0 errata, and everything else at zero.

#### Why it happened

A repository only holds packages after it's pointed at a real web address and told to actually download ("sync") from it. Simply creating it doesn't pull anything in by itself.

#### How we fixed it

We set the Upstream URL on the repository to a real, reachable source (for example, a public CentOS Stream or EPEL mirror, since a full official Red Hat subscription wasn't available), then triggered a sync from Content > Products.

**If this happens again:** After creating a repository, always check its content counts. If everything reads 0, the Upstream URL is either missing or hasn't been synced yet.

### 15. A repository sync failing partway through

## What we were trying to do

Download a large repository (BaseOS) completely into Foreman.

## What went wrong

The sync failed partway with a "ContentLengthError", meaning a file download got cut off before it finished completely.

## Why it happened

This is usually just a flaky internet connection or an overloaded mirror server, not a real configuration mistake — large syncs occasionally lose a connection partway through.

## How we fixed it

We simply retried the sync, which succeeded the second time, since it re-downloaded only the missing pieces rather than starting over from scratch.

**If this happens again:** If a sync fails with a content-length or truncated-download type error, just retry it first — this is usually temporary. If it keeps failing, try a different, more reliable mirror address, or check available disk space on the server.

# 16. Content Views moved to a different menu

## What we were trying to do

Find the "Content Views" section to bundle the newly synced repository.

## What went wrong

The left-hand menu had no "Content Views" item under the main Content section at all.

## Why it happened

The newer version of Foreman/Katello (installed via the container-based method) reorganized its menu; Content Views was moved under a new "Lifecycle" submenu instead of sitting directly under Content.

## How we fixed it

We expanded the "Lifecycle" section in the sidebar, which revealed Content Views, Lifecycle Environments, and Activation Keys all grouped together there.

**If this happens again:** If a menu item described in an older guide seems to be missing, check nearby collapsible submenus first — software interfaces get reorganized between versions fairly often.

## Part 3: Registering a Client Server

Connecting an actual Linux machine to the new patch management server.

### 17. Root password not accepted

#### What we were trying to do

Get administrator access on the client machine to run the registration command.

#### What went wrong

Typing what seemed like the correct root password kept failing with "Authentication failure".

#### Why it happened

This is most often caused by Caps Lock being on, a keyboard layout mismatch, invisible extra characters from copy-pasting, or simply not knowing/remembering the actual root password if it was never explicitly set.

#### How we fixed it

Since the regular user account already had administrator (sudo) rights, we bypassed the root password entirely:

```
sudo -i
```

**If this happens again:** You rarely need the actual root password if your own user account has sudo rights. Use "sudo -i" to get an administrator shell using your own account's password instead.

### 18. Registration failing as "unauthorized"

#### What we were trying to do

Register the client machine to Foreman using an activation key.

#### What went wrong

The registration command returned "ERROR: unauthorized".

#### Why it happened

This can happen for several reasons: the activation key name doesn't exactly match one that actually exists, it belongs to a different organization than expected, or (as turned out to be the case here) a short-lived authorization code in the command had a separate, unrelated problem (see the next section).

#### How we fixed it

We first confirmed the activation key genuinely existed with the exact name being used, by checking Content > Activation Keys in the web interface, and used the interface's own "Register Host" page to generate the exact, correct command instead of typing it by hand.



**If this happens again:** Always double-check an activation key's exact name and organization in the web interface first. Where possible, use the interface's own "generate command" feature instead of typing a registration URL from memory, since it fills in the correct IDs automatically.

## 19. Registration blocked by a security certificate check

### What we were trying to do

Run the exact registration command generated by the web interface.

### What went wrong

The command failed with "SSL certificate problem: self-signed certificate in certificate chain".

### Why it happened

Foreman's website uses a self-made (self-signed) security certificate by default, since it wasn't issued by a widely recognized outside authority. Command-line tools refuse to trust this automatically, the same way a browser shows a warning page for it.

### How we fixed it

We added a flag telling the tool to trust the connection anyway, which is an acceptable shortcut for a private homelab:

```
curl -k --silent --show-error '...' # -k means: trust this connection anyway
```

**If this happens again:** If a command-line tool complains about a self-signed certificate, and you're confident the server is genuinely your own trusted machine, adding `-k` (or `--insecure`) to the command is an acceptable homelab-level fix. For a more permanent, more correct fix later, you can install the server's own security certificate onto client machines so they trust it properly instead of bypassing the check every time.

## 20. Client thinking it's already registered

### What we were trying to do

Re-run the registration command after fixing the certificate issue.

### What went wrong

The command reported "This system is already registered", but the Foreman website's Hosts list still showed no hosts at all — a mismatch between what the client believed and what the server had on record.

### Why it happened

An earlier, partially-failed registration attempt (before the certificate fix) had left some local leftover registration information on the client, even though it never actually succeeded on Foreman's side.

### How we fixed it

We cleared that stale local information first, then registered again fresh:

```
subscription-manager unregister  
subscription-manager clean
```

Then re-ran the same registration command used before (with the certificate fix included).

**If this happens again:** If a client claims to already be registered but doesn't show up on the server, clear its local registration state first ("subscription-manager unregister" and "clean"), then register again from scratch. Also note that the authorization code in a generated registration command is often short-lived — if it's been a while since it was generated, get a fresh one from the web interface rather than reusing an old one.

## General lessons for next time

- Set a shell variable and use it right away, in the same terminal window — don't switch windows in between, or it disappears.
- On real RHEL, EPEL needs its full web address, not just a short package name.
- The phrase "filtered out by modular filtering" almost always means two software versions are in conflict. Check with "dnf module list" and use "switch-to" to move between them cleanly.
- Never interrupt a long-running installer with Ctrl+C unless it's genuinely stuck. Use a second window to check progress instead.
- If a required older software version has disappeared completely (not just "disabled"), you can't go backward — look for a newer, currently-supported approach instead.
- If an error names an exact version limit and switching versions doesn't fix it, that's usually a real, current limitation. Check official docs/forums for the currently recommended method.
- A container-based installer is often more reliable than a traditional package-based one when fighting operating-system version mismatches.
- After a failed install attempt, if a database complains its data folder is invalid, it's usually safe (with no real data yet) to clear that folder and let it start fresh.
- If a background service shows as "running" but never appears in your container list, check whether a plain, non-container copy was installed separately and is quietly taking priority.
- Fix warnings in service logs immediately, even if things currently work — they often predict a failure you'll hit soon after.
- Save any auto-generated admin password into a password manager right away — it's often shown only once.
- A custom ".local" address needs a manual hosts file entry on every computer that needs to reach it.
- You rarely need the true root password if your account has sudo rights — use "sudo -i" instead.
- Always confirm an activation key's exact name and organization in the web interface, and use the interface's own "generate command" option instead of typing registration URLs from memory.
- A self-signed certificate warning from a command-line tool can be bypassed with -k/--insecure for a private homelab server you trust.
- If a client claims to be "already registered" but doesn't appear on the server, clear its local registration state and register again from scratch; also request a fresh authorization code if it's been a while since the last one was generated.